Course: Enterprise Real-Time Data Engineering
Instructor: Mohamed Fouad Fakhruldeen (Fakhr.me)
Course Portal: fakhr.me/KafkaSpark
Code & Labs Repository: github.com/fakhruldeen/KafkaSpark

Session S02: Hands-On Lab Walkthrough

Topic Provisioning, Partition Mechanics, Physical Segment Anatomy & High-Availability Chaos Drills

Lab Overview & Engineering Objectives

In this hands-on engineering lab, you will explore the physical and distributed mechanics of Apache Kafka topics, partitions, and replication. You will operate directly against a multi-broker KRaft cluster running in Docker, inspect binary log segments on disk, verify deterministic MurmurHash2 key routing, and perform real-time Chaos Engineering drills (killing broker instances and witnessing instant leader failover).

By the end of this lab, you will be able to:
1. **Provision multi-partition, replicated topics** using production-grade quorum configs (`min.insync.replicas`).
2. **Decode topic metadata output**: understand the exact role of Partition Leaders, Replicas, and the In-Sync Replicas (ISR) set.
3. **Inspect the physical filesystem**: examine `.log` , `.index` , `.timeindex` , and checkpoint files inside container storage.
4. **Dump raw binary Kafka records** using the official `kafka-dump-log.sh` utility.
5. **Mathematically prove deterministic key routing**: observe why identical keys always land on identical partitions.
6. **Tune topic configurations on a running cluster** using `kafka-configs.sh` without downtime.
7. **Execute intentional fault injection (Chaos Engineering)**: kill a leader node, verify sub-second failover, produce during degraded quorum, and observe automatic ISR self-healing upon node recovery.
8. **Validate all achievements** via the automated verification script (`./verify_s02.sh`).

📖 Master Parameter & CLI Argument Encyclopedia

Before executing commands, master the exact technical mechanics of every CLI flag used in this lab:

1. `kafka-topics.sh` Parameters

Parameter / Flag	Type	Purpose & Under-the-Hood Mechanics
`--bootstrap-server <host:port>`	Network URI	Connects to an initial broker (`kafka1:9092` inside Docker, or `localhost:9092` from host). The client sends an `ApiVersions` request and downloads the complete metadata of all brokers in the cluster.
`--create`	Action	Instructs the KRaft Controller to allocate partition metadata, assign replica sets across brokers, and persist the record in the `@metadata` partition.
`--topic <string>`	Name	The unique logical identifier of the topic. Enterprise standard: `<domain>.<dataset>.<environment>` (e.g. `nile.inventory-events.prod`).
`--partitions <int>`	Parallelism	Number of physical log partitions. This is the **fundamental unit of scalability and consumer parallelism** in Kafka. A topic with N partitions can be consumed in parallel by at most N consumers in a group.
`--replication-factor <int>`	Redundancy	Total number of physical copies of each partition across distinct brokers. An $RF = 3$ topic stores 1 Leader and 2 Follower copies. **Rule:** $RF \leq$ Total Active Brokers.
`--config min.insync.replicas=<int>`	Durability Gate	Minimum number of replicas that must successfully write to disk before an acknowledgment (`ACK`) is returned to a producer using `acks=all` . If the active ISR count drops below this value, write requests are rejected with `NotEnoughReplicasException` .
`--config retention.ms=<long>`	Lifecycle	Maximum retention time in milliseconds before log cleaner or deleter deletes older segments (e.g. `604800000` = 7 days, or `86400000` = 24 hours).
`--config segment.bytes=<long>`	Physical File	Maximum size of an individual `.log` segment file (default is `1073741824` = 1 GB). When this limit is reached, the active segment is rolled (closed) and a new segment is allocated.
`--config unclean.leader.election.enable`	Consensus	Boolean flag (`true` / `false`). If `false` (default & enterprise standard), an out-of-sync replica that is NOT in the ISR can NEVER be elected Leader, preventing silent data loss.
`--describe`	Action	Queries metadata for partition leadership, replica distribution, ISR health, and overridden topic-level configs.
`--alter`	Action	Modifies partition count (can only increase, never decrease) or dynamic topic configs.

2. `kafka-console-producer.sh` & `kafka-console-consumer.sh` Parameters

Parameter / Flag	Component	Purpose & Under-the-Hood Mechanics
`--property "parse.key=true"`	Producer	Instructs the line reader to split input strings into a distinct **Key** and **Value**.
`--property "key.separator=:"`	Producer	Defines the delimiter separating the key from the value payload (e.g., `KEY:VALUE`).
`--producer-property acks=all`	Producer	Instructs the producer to wait for confirmation from **all active ISR replicas** before considering the send successful.
`--from-beginning`	Consumer	Instructs the consumer to reset its read pointer to the earliest available offset (0) instead of waiting for future messages.
`--property print.key=true`	Consumer	Prints the message key preceding the payload.
`--property print.partition=true`	Consumer	Prints the physical partition index $(0, 1, 2)$ from which the event was read.
`--property print.offset=true`	Consumer	Prints the 64-bit integer sequential offset of the record.
`--property print.timestamp=true`	Consumer	Prints the epoch millisecond timestamp assigned by the client or broker.

🛠️ Step-by-Step Lab Execution

Step 1: Pre-Flight Cluster Verification

Verify that your 3-broker KRaft cluster is up and healthy:

```
cd ~/Desktop/Work/Training/Spark/00_Lab
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}" | grep kafka
```

Expected Terminal Output:

```
kafka1   Up (healthy)   0.0.0.0:9092→9094/tcp
kafka2   Up (healthy)   0.0.0.0:9093→9094/tcp
kafka3   Up (healthy)   0.0.0.0:9094→9094/tcp
kafka-ui Up             0.0.0.0:8080→8080/tcp
```

Step 2: Create an Enterprise Replicated Topic

Create a new topic named `nile.inventory-events` with **3 partitions**, a **replication factor of 3**, and strict durability constraints:

```
docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server kafka1:9092 \
  --create \
  --topic nile.inventory-events \
  --partitions 3 \
  --replication-factor 3 \
  --config min.insync.replicas=2 \
  --config segment.bytes=10485760 \
  --config retention.ms=86400000
```

> **Parameter Check:**
> - `partitions 3`: Allows 3 parallel consumer workers.
> - `replication-factor 3`: All 3 brokers maintain a copy.
> - `min.insync.replicas=2`: Requires at least 2 nodes to acknowledge writes.
> - `segment.bytes=10485760`: Segment roll threshold lowered to 10 MB for testing.
> - `retention.ms=86400000`: Retain logs for 24 hours.

Step 3: Inspect Physical Partition Layout & Quorum Metadata

Run the `--describe` command to analyze partition placement across the cluster:

```
docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server kafka1:9092 \
  --describe \
  --topic nile.inventory-events
```

Analyze the Output:

```
Topic: nile.inventory-events    TopicId: ...    PartitionCount: 3   ReplicationFactor: 3
Configs: min.insync.replicas=2,segment.bytes=10485760,retention.ms=86400000
    Topic: nile.inventory-events    Partition: 0    Leader: 1   Replicas: 1,2,3 Isr: 1,2,3
    Topic: nile.inventory-events    Partition: 1    Leader: 2   Replicas: 2,3,1 Isr: 2,3,1
    Topic: nile.inventory-events    Partition: 2    Leader: 3   Replicas: 3,1,2 Isr: 3,1,2
```

Key Architectural Insights:

1. **Balanced Leadership:** Partition 0 is led by Broker 1, Partition 1 by Broker 2, and Partition 2 by Broker 3. This evenly distributes network ingress/egress.
2. **Replicas Set:** Shows the ordered assigned replica brokers.
3. **ISR (In-Sync Replicas):** All 3 brokers currently have replicated 100% of committed log offsets.

Step 4: Physical Disk Inspection inside Broker Storage

Let's look under the hood at the actual Linux filesystem where Kafka writes files.
Execute a directory listing inside the data volume of `kafka1`:

```
docker exec -it kafka1 ls -la /var/lib/kafka/data/nile.inventory-events-0/
```

Expected Filesystem Output:

```
-rw-r--r-- 1 appuser appuser 10485760 ... 00000000000000000000.index
-rw-r--r-- 1 appuser appuser        0 ... 00000000000000000000.log
-rw-r--r-- 1 appuser appuser 10485760 ... 00000000000000000000.timeindex
-rw-r--r-- 1 appuser appuser       10 ... leader-epoch-checkpoint
-rw-r--r-- 1 appuser appuser        8 ... partition.metadata
```

Deep Engineering Breakdown:
- `00000000000000000000.log`: The active append-only data file storing raw batches.
- `00000000000000000000.index`: The sparse index file mapping logical offsets to physical byte positions on disk.
- `00000000000000000000.timeindex`: Maps message timestamps to logical offsets for time-based lookups.
- `leader-epoch-checkpoint`: Monotonically increasing epoch counter used by KRaft to prevent log divergence during failovers.

Step 5: Dump Raw Log Segments with `kafka-dump-log.sh`

Let's inspect how Kafka encodes data inside `.log` files using Kafka's internal forensic tool:

```
docker exec -it kafka1 /opt/kafka/bin/kafka-dump-log.sh \
  --files /var/lib/kafka/data/nile.inventory-events-0/00000000000000000000.log \
  --print-data-log \
  --deep-iteration
```

> **Tip:** Currently this segment is empty because no records have been produced yet. We will run this again after Step 6!

Step 6: Verify Deterministic MurmurHash2 Partition Key Routing

Open **Terminal Window 1** (The Consumer with Metadata Properties):

```
docker exec -it kafka1 /opt/kafka/bin/kafka-console-consumer.sh \
  --bootstrap-server kafka1:9092 \
  --topic nile.inventory-events \
  --from-beginning \
  --property print.key=true \
  --property print.partition=true \
  --property print.offset=true \
  --property print.timestamp=true \
  --property key.separator=" | "
```

Open **Terminal Window 2** (The Keyed Console Producer):

```
docker exec -it kafka1 /opt/kafka/bin/kafka-console-producer.sh \
  --bootstrap-server kafka1:9092 \
  --topic nile.inventory-events \
  --property "parse.key=true" \
  --property "key.separator=:" \
  --producer-property acks=all
```

Now copy and paste these retail inventory transactions into **Terminal Window 2**:

```
SKU-APPLE-15:{"item": "iPhone 15 Pro", "qty": -1, "store": "Nasr City"}
SKU-SAMS-S24:{"item": "Galaxy S24", "qty": -2, "store": "Zamalek"}
SKU-APPLE-15:{"item": "iPhone 15 Pro", "qty": +50, "store": "Warehouse"}
SKU-DELL-XPS:{"item": "Dell XPS 15", "qty": -1, "store": "Maadi"}
SKU-APPLE-15:{"item": "iPhone 15 Pro", "qty": -3, "store": "Alexandria"}
SKU-SAMS-S24:{"item": "Galaxy S24", "qty": +20, "store": "Warehouse"}
```

🔍 Analyze the Consumer Terminal Output:

Observe the partition numbers displayed on the consumer terminal:
- Notice that **every single message** with key `SKU-APPLE-15` lands on the **exact same partition** (e.g. `Partition: 0` or `1`) in sequential offset order ($0, 1, 2 \ldots$).
- Notice that `SKU-SAMS-S24` lands consistently on its own assigned partition.
- **Why?** Because Kafka computes:

$$\text{Partition} = \text{toPositive}(\text{MurmurHash2}(\text{keyBytes})) \pmod 3$$

- **Business Guarantee:** Out-of-order inventory updates are mathematically impossible when a key is used!

Now re-run the log dumper to see the binary record on disk:

```
docker exec -it kafka1 /opt/kafka/bin/kafka-dump-log.sh \
  --files /var/lib/kafka/data/nile.inventory-events-0/00000000000000000000.log \
  --print-data-log
```

Notice the `CreateTime`, `baseOffset`, `magic: 2`, `CRC`, `key`, and `payload` displayed directly from disk!

Step 7: Dynamic Topic Configuration Tuning with `kafka-configs.sh`

In enterprise environments, you never restart brokers to update topic configurations.
Alter the retention time to 48 hours (`172800000 ms`) dynamically:

```
docker exec -it kafka1 /opt/kafka/bin/kafka-configs.sh \
  --bootstrap-server kafka1:9092 \
  --entity-type topics \
  --entity-name nile.inventory-events \
  --alter \
  --add-config retention.ms=172800000
```

Verify that the dynamic config was applied:

```
docker exec -it kafka1 /opt/kafka/bin/kafka-configs.sh \
  --bootstrap-server kafka1:9092 \
  --entity-type topics \
  --entity-name nile.inventory-events \
  --describe
```

Output:

```
Dynamic configs for topic nile.inventory-events are:
  retention.ms=172800000 sensitive=false synonyms={DYNAMIC_TOPIC_CONFIG:retention.ms=1728000
00}
```

Step 8: Live Chaos Engineering - Kill a Leader Broker

Let's test what happens when hardware catches fire in production.

1. Check who is the Leader for Partition 1 (from Step 3, let's assume it is Broker 2 / `kafka2`).
2. Simulate a catastrophic kernel panic or machine failure by stopping `kafka2` :
   `bash`
   `docker stop kafka2`
3. Immediately query the topic metadata from surviving `kafka1` :
   `bash`
   `docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh \`
   `--bootstrap-server kafka1:9092 \`
   `--describe \`
   `--topic nile.inventory-events`

Observe the Failover Reality:

```
Topic: nile.inventory-events    Partition: 1    Leader: 3   Replicas: 2,3,1 Isr: 3,1
```

- **Instant Failover:** Leader for Partition 1 was promoted to Broker 3 in milliseconds.
- **Degraded ISR:** Broker 2 was automatically expelled from the `Isr` set (`Isr: 3,1`).
- **High Availability:** Since `min.insync.replicas=2` and 2 healthy brokers remain (`3,1`), the topic is **100% available for both reads and writes!**

4. Send a new transaction while Broker 2 is dead:

```
bash
   echo "SKU-APPLE-15:{\"item\":\"iPhone 15 Pro\",\"delta\":-1,\"test\":\"during-outage\"}"
| \
   docker exec -i kafka1 /opt/kafka/bin/kafka-console-producer.sh \
     --bootstrap-server kafka1:9092 \
     --topic nile.inventory-events \
     --property "parse.key=true" \
     --property "key.separator=:" \
     --producer-property acks=all
```

The message is accepted and confirmed immediately!

Step 9: Broker Recovery and Self-Healing ISR Rejoin

Now bring Broker 2 back to life:

```
docker start kafka2
sleep 5
```

Re-describe the topic to observe self-healing:

```
docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server kafka1:9092 \
  --describe \
  --topic nile.inventory-events
```

Expected Result:

```
Topic: nile.inventory-events    Partition: 1    Leader: 3   Replicas: 2,3,1 Isr: 3,1,2
```

Broker 2 fetched all missing offsets from the active leader in the background and was automatically restored into the **ISR quorum (`Isr: 3,1,2`)**! Zero data loss, zero human intervention!

Step 10: Clean Up & Run Automated Verification Gate

Delete the temporary lab topic:

```
docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server kafka1:9092 \
  --delete --topic nile.inventory-events
```

Now execute the automated S02 verification script from `00_Lab`:

```
cd ~/Desktop/Work/Training/Spark/00_Lab
./verify_s02.sh
```

Expected 100% PASS Output:

```
================================================
 Running S02 Lab Verification Gate...
================================================
[1/6] Checking Kafka cluster reachability... PASS
[2/6] Verifying 3-partition, RF 3 topic creation... PASS (3 Partitions, RF 3, min.insync.rep
licas=2)
[3/6] Testing deterministic MurmurHash2 key routing... PASS
[4/6] Verifying key-to-partition affinity... PASS (KEY_A strictly bound to Partition 0)
[5/6] Chaos Test: Broker failover & leader election... PASS (Active leader promoted to survi
ving broker in < 1s)
[6/6] Chaos Test: Broker recovery & ISR self-healing... PASS (kafka2 caught up and rejoined
ISR successfully)
================================================
★ PASS: S02 Lab Requirements 100% Satisfied! ★
================================================
```

🌟 Gold Stretch Challenge: Log Compaction Verification

Distinguished Engineer Challenge: If you complete the core lab before time expires, test Kafka's database-like log compaction:

1. Create a compacted topic with aggressive segment cleaner settings:

```
bash
   docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh \
     --bootstrap-server kafka1:9092 \
     --create --topic nile.user-status \
     --partitions 3 --replication-factor 3 \
     --config cleanup.policy=compact \
     --config min.cleanable.dirty.ratio=0.01 \
     --config segment.ms=5000
```

2. Produce multiple updates for the same key:

```
text
   USER_101:{"status": "PENDING"}
   USER_101:{"status": "ACTIVE"}
   USER_101:{"status": "SUSPENDED"}
```

3. Observe with `kafka-console-consumer.sh` how the log cleaner thread compacts historical records, keeping only the latest known state (`SUSPENDED`) per key.